

Interpolation for the simply-typed λ -calculus

Meven LENNON-BERTRAND

Alexis SAURIN

November 26, 2025

Rewriting Čubrić [Čub94] in a modern type-theoretic language.

1 Context

Čubrić [Čub94] proves interpolation for the internal language of biCCC, that is, STLC with at least pair, unit, sum, and empty types, all with η -laws (including for positive connectives!), plus arbitrary base types, constants, and equations.

He really needs commuting conversions for $+$ and $\perp$, apparently because of some subformula property. I wonder how this looks like in our setting...

2 Definitions

2.1 λ -calculus

Definition We work with STLC, which we take to always have unit, product and function types (*i.e.* this is the internal language of CCC). We moreover work with respect to a theory Θ .

Definition 1 – Language

A *language* Λ consists of:

- a set of base types $\mathcal{B} \in \mathbf{Set}$;
- a family of typed constants indexed by types $\mathcal{C} \in \mathbf{Ty}_{\mathcal{B}} \rightarrow \mathbf{Set}$;

In the above, $\mathbf{Ty}_{\mathcal{B}}$ is the set of types generated by $\mathcal{B}$ which will be defined in Fig. 1. If $\Lambda = (\mathcal{B}, \mathcal{C})$ we write $\mathbf{Ty}_{\Lambda}$ for $\mathbf{Ty}_{\mathcal{B}}$, and we write $\emptyset$ for the empty language, *i.e.* that where the sets of base types and constants are empty.

Technically, we should interleave the definition of types generated by a set of base types with that of a language, but that's annoying (and becomes impossible anyway once you move to dependent type theory, see *e.g.* Bezem et al. [Bez+21] for the issue and an elegant solution).

Definition 2 – Theory

A *theory* Θ consists of:

- a language Λ ;

$\boxed{\vdash \Gamma}$ Contexts $\Gamma, \Delta, \dots \in \text{Ctx}_\Lambda$

$$\text{EMP} \frac{}{\vdash \cdot} \quad \text{EXT} \frac{\vdash \Gamma \quad \vdash A}{\vdash \Gamma.(x:A)}$$

$\boxed{\vdash A}$ Types $A, B, \dots \in \text{Ty}_\Lambda$

$$\text{BASE} \frac{b \in \mathcal{B}}{\vdash b} \quad \text{UNIT} \frac{}{\vdash \top} \quad \text{PROD} \frac{\vdash A \quad \vdash B}{\vdash A \times B} \quad \text{FUN} \frac{\vdash A \quad \vdash B}{\vdash A \rightarrow B}$$

$\boxed{\Gamma \vdash t : A}$ Terms $t, u, \dots \in \text{Tm}_\Lambda(A)$

$$\begin{array}{c} \text{CST} \frac{c \in \mathcal{C}(A)}{\Gamma \vdash c : A} \quad \text{VAR} \frac{(x:A) \in \Gamma}{\Gamma \vdash x : A} \quad \text{STAR} \frac{}{\Gamma \vdash * : \top} \\[10pt] \text{PAIR} \frac{\Gamma \vdash t : A \quad \Gamma \vdash u : B}{\Gamma \vdash (t, u) : A \times B} \quad \text{PROJ1} \frac{\Gamma \vdash p : A \times B}{\Gamma \vdash p_1 : A} \quad \text{PROJ2} \frac{\Gamma \vdash p : A \times B}{\Gamma \vdash p_2 : B} \\[10pt] \text{LAM} \frac{\Gamma.(x:A) \vdash t : B}{\Gamma \vdash \lambda x.t : A \rightarrow B} \quad \text{APP} \frac{\Gamma \vdash t : A \rightarrow B \quad \Gamma \vdash u : A}{\Gamma \vdash t u : B} \end{array}$$

Figure 1: Types and terms of the simply-typed λ -calculus (with respect to a language $\Lambda = (\mathcal{B}, \mathcal{C})$)

- a set of equations $\mathcal{E} : (A : \text{Ty}_\Lambda) \rightarrow \mathcal{P}(\text{Tm}_\Lambda(A)^2)$.
- $\text{Tm}_\Lambda(A)$ is the set of terms of type A generated by the language Λ , to be defined in Fig. 1.
If $\Theta = (\Lambda, \mathcal{E})$ we write Ty_Θ for Ty_Λ , and similarly for Tm .

The definition of types and terms is given in Fig. 1. MLB ►Technically, we should specify the substitution calculus...◀

The equational theory is in Fig. 2. We omit congruence equations.

Extra operations

Definition 3 – Context partition

The context partition relation is defined as a (partial) relation $\Gamma = \Gamma_1 \cup \Gamma_2$ by

$$\frac{}{\cdot = \cdot \cup \cdot} \quad \frac{\Gamma = \Gamma_1 \cup \Gamma_2}{\Gamma.(x:A) = \Gamma_1.(x:A) \cup \Gamma_2} \quad \frac{\Gamma = \Gamma_1 \cup \Gamma_2}{\Gamma.(x:A) = \Gamma_1 \cup \Gamma_2.(x:A)}$$

Definition 4 – Reduction

Reduction is written $\rightsquigarrow$. MLB ►What sort of reduction is this?◀

$\boxed{\Gamma \vdash t \equiv u : A}$ Equality of terms		
$\text{REFL} \frac{\Gamma \vdash t : A}{\Gamma \vdash t \equiv t : A}$	$\text{SYM} \frac{\Gamma \vdash t \equiv u : A}{\Gamma \vdash u \equiv t : A}$	$\text{REFL} \frac{\Gamma \vdash t \equiv u : A \quad \Gamma \vdash t \equiv v : A}{\Gamma \vdash t \equiv v : A}$
congruence/substitution rule		
$\text{EQ} \frac{(t, u) \in \mathcal{E}(A)}{\Gamma \vdash t \equiv u : A}$		$\text{UNIT}\eta \frac{\Gamma \vdash t : \top}{\Gamma \vdash t \equiv * : \top}$
$\text{PAIR}\beta \frac{\Gamma \vdash t : A \quad \Gamma \vdash u : B}{\Gamma \vdash (t, u).1 \equiv t : A \quad \Gamma \vdash (t, u).2 \equiv u : B}$		$\text{PAIR}\eta \frac{\Gamma \vdash p : A \times B}{\Gamma \vdash p \equiv (p.1, p.2) : A \times B}$
$\text{FUN}\beta \frac{\Gamma, x : A \vdash t : B \quad \Gamma \vdash u : A}{\Gamma \vdash (\lambda x. t) u \equiv t[x := u] : B}$		$\text{FUN}\eta \frac{\Gamma \vdash f : A \rightarrow B}{\Gamma \vdash f \equiv \lambda x. (f x) : A \rightarrow B}$

Figure 2: Equations of the simply-typed λ -calculus (with respect to a theory $\Theta = (\Lambda, \mathcal{E})$)

2.2 Bidirectional typing

We can characterise those terms which have a nice “information flow” by using bidirectional typing. Incidentally, this characterises normal/neutral forms – a very deep mystery, but a useful one. Contexts and types are the same as in Fig. 1 above, the term judgments are in Fig. 3.

There is an embedding of bidirectional terms into undirected ones, which we call erasure and write

$$\begin{aligned} |\cdot| : \text{Nf}_\Lambda(A) &\rightarrow \text{Tm}_\Lambda(A) \\ |\cdot| : \text{Ne}_\Lambda(A) &\rightarrow \text{Tm}_\Lambda(A) \end{aligned}$$

Both are entirely structural, up to $|\underline{t}| = |t|$. We generally leave them implicit.

Theorem 5 – Normalisation

If Σ is a theory with no equations, given a term $t \in \text{Tm}_\Sigma(A)$, there exists a unique $n \in \text{Nf}_\Sigma(A)$ such that $t = |n|$.

2.3 Evaluation context

Another way to capture neutrals, which looks a bit closer to what is happening in Čubrić’s [Čub94] proof, is to use *evaluation stacks*, see Fig. 4. A stack represents a succession of elimination forms. Any neutral can be decomposed into a variable plugged into such a stack, which captures the succession of eliminations which compose the neutral. The important difference is that stacks are “inside out”, *i.e.* the top of the stack corresponds to the innermost elimination of the neutral. This seems to be closer to what happens in Čubrić [Čub94].

$\boxed{\Gamma \vdash t \triangleleft A}$ Checking/normal terms $t, u, \dots \in \mathbf{Nf}_\Lambda(\Gamma, A)$

$$\begin{array}{c}
\text{STAR} \frac{}{\Gamma \vdash * \triangleleft \top} \quad \text{PAIR} \frac{\Gamma \vdash t \triangleleft A \quad \Gamma \vdash u \triangleleft B}{\Gamma \vdash (t, u) \triangleleft A \times B} \quad \text{LAM} \frac{\Gamma, x: A \vdash t \triangleleft B}{\Gamma \vdash \lambda x. t \triangleleft A \rightarrow B} \\
\text{DEMOTE} \frac{\Gamma \vdash t \triangleright A \quad A = B}{\Gamma \vdash \underline{t} \triangleleft B}
\end{array}$$

$\boxed{\Gamma \vdash t \triangleright A}$ Inferring/neutral terms $t, u, \dots \in \mathbf{Ne}_\Lambda(\Gamma, A)$

$$\begin{array}{c}
\text{CST} \frac{c \in \mathcal{C}(A)}{\Gamma \vdash c \triangleright A} \quad \boxed{\text{MLB}} \triangleright \text{Is this inferring or checking?} \triangleleft \quad \text{VAR} \frac{(x: A) \in \Gamma}{\Gamma \vdash x \triangleright A} \\
\text{PROJ1} \frac{\Gamma \vdash p \triangleright A \times B}{\Gamma \vdash \pi_1(p) \triangleright A} \quad \text{PROJ2} \frac{\Gamma \vdash p \triangleright A \times B}{\Gamma \vdash \pi_2(p) \triangleright B} \quad \text{APP} \frac{\Gamma \vdash t \triangleright A \rightarrow B \quad \Gamma \vdash u \triangleleft A}{\Gamma \vdash t u \triangleright B}
\end{array}$$

Figure 3: Bidirectional typing for the simply-typed λ -calculus (with respect to a language $\Lambda = (\mathcal{B}, \mathcal{C})$)

$\boxed{\Gamma \mid A \vdash \pi \triangleright B}$ Stacks $\pi, \dots \in \mathbf{Sta}_\Lambda(\Gamma, A, B)$

$$\begin{array}{c}
\text{NIL} \frac{}{\Gamma \mid T \vdash [] \triangleright T} \quad \text{PROJi} \frac{\Gamma \mid A_i \vdash \pi \triangleright T}{\Gamma \mid A_1 \times A_2 \vdash \square_i \pi \triangleright T} \quad \text{APP} \frac{\Gamma \mid B \vdash \pi \triangleright T \quad \Gamma \vdash u \triangleleft A}{\Gamma \mid A \rightarrow B \vdash \square u \pi \triangleright T}
\end{array}$$

Figure 4: Evaluation stacks for the simply-typed λ -calculus

Definition 6 – Stack operations

Given a stack and a neutral, we can always plug the latter into the former:

$$\begin{aligned} \llbracket _ \rrbracket : \text{Sta}(\Gamma, A, B) &\rightarrow \text{Ne}(\Gamma, A) \rightarrow \text{Ne}(\Gamma, B) \\ \llbracket _ \rrbracket [t] &:= t \\ (\llbracket _ \rrbracket_i :: \pi)[t] &:= \pi[t_i] \\ (\llbracket _ \rrbracket u :: \pi)[t] &:= \pi[t \ u] \end{aligned}$$

Conversely, any neutral can be (uniquely) decomposed into a form $\pi[x]$.

3 Interpolation

Definition 7 – Polarities

Let $A \in \text{Ty}_{\mathcal{B}}$. We define $A^+ \subseteq \mathcal{B}$ (resp. $A^- \subseteq \mathcal{B}$) by induction:

$$\begin{aligned} b^+ &:= \{b\} \\ b^- &:= \emptyset \\ (A \times B)^p &:= A^p \cup B^p \\ (A \rightarrow B)^p &:= A^{-p} \cup B^p \\ \top^p &:= \emptyset \end{aligned}$$

This is lifted to contexts as $(\Gamma.(x:A))^p := \Gamma^p \cup A^p$.

Main theorem The first version of interpolation works with constant-free theories, *i.e.* those of the form $((\mathcal{B}, \emptyset), \emptyset)$.

Theorem 8 – Interpolation – constant-free

Let Σ be a constant-free theory, and assume a partitioned context $\Gamma = \Gamma_s \cup \Gamma_t$.

Given any normal term $t \in \text{Nf}_{\Sigma}(\Gamma, T)$, there is a type $M \in \text{Ty}_{\Sigma}$ and terms $l \in \text{Tm}_{\Sigma}(\Gamma_s, M)$ and $r \in \text{Tm}_{\Sigma}(\Gamma_t, (x:M), T)$ such that

- $\Gamma \vdash r[x := l] \equiv t : T$;
- $M^+ \subseteq \Gamma_s^+ \cap (\Gamma_t^- \cup T^+)$ and $M^- \subseteq \Gamma_s^- \cap (\Gamma_t^+ \cup T^-)$.

Moreover, given any neutral term $t \in \text{Ne}_{\Sigma}(\Gamma, T)$, there is a type $M \in \text{Ty}_{\Sigma}$ such that either:

1. “ T is a source type”, *i.e.* we have terms $l \in \text{Tm}_{\Sigma}(\Gamma_t, M)$ and $r \in \text{Tm}_{\Sigma}(\Gamma_s, (x:M), T)$ such that
 - $\Gamma \vdash r[x := l] \equiv t : T$,
 - $T^+ \subseteq \Gamma_s^+$ and $T^- \subseteq \Gamma_s^-$,
 - $M^+ \subseteq \Gamma_s^+ \cap \Gamma_t^-$ and $M^- \subseteq \Gamma_s^- \cap \Gamma_t^+$;
2. or “ T is a target type”, *i.e.* we have terms $l \in \text{Tm}_{\Sigma}(\Gamma_s, M)$ and $r \in \text{Tm}_{\Sigma}(\Gamma_t, (x:M), T)$ such that
 - $\Gamma \vdash r[x := l] \equiv t : T$,
 - $T^+ \subseteq \Gamma_t^+$ and $T^- \subseteq \Gamma_t^-$,
 - $M^+ \subseteq \Gamma_s^+ \cap \Gamma_t^-$ and $M^- \subseteq \Gamma_s^- \cap \Gamma_t^+$.

Proof

By induction on the definition of **Ne** / **Nf**.

The **Nf** cases are rather direct.

STAR: T is $\top$, so we can take $M = \top$, and $l = r = \star$.

PAIR: T is $T_1 \times T_2$, t is (t_1, t_2) . By induction, we obtain $l_i \in \text{Tm}(\Gamma_s, M_i)$ and $r_i \in \text{Tm}(\Gamma_t.(x: M_i), T_i)$.

We can then take $M := M_1 \times M_2$ and

$$\begin{array}{lcl} \Gamma_s \vdash l := (l_1, l_2) & & : M_1 \times M_2 \\ \Gamma_t.(x: M_1 \times M_2) \vdash r := (r_1[y := y_1], r_2[y := y_2]) & & : T_1 \times T_2 \end{array}$$

LAM: We extend the *target* context with the extra variable (this is why we need this more complicated formulation with a context split!) and by induction hypothesis, we obtain $l' \in \text{Tm}(\Gamma_s, M)$ and $r' \in \text{Tm}(\Gamma_t.(x: A).(z: M))$, and we can take

$$\begin{array}{lcl} \Gamma_s \vdash l := l' & & : M \\ \Gamma_t.(z: M) \vdash r := \lambda x.r' & & : A \rightarrow B \end{array}$$

The **Ne** cases are more interesting. For each rule, we have two subcases, depending on the source/target induction hypothesis for the main sub-neutral, or on whether the variable in is the source or target context.

VAR/1: We have $(x : A) \in \Gamma_s$. Then we take $M := \top$

$$\begin{array}{lcl} \Gamma_t \vdash l := \star & & : M \\ \Gamma_s.(x: A) \vdash r := x & & : A \end{array}$$

VAR/2: We have $(x : A) \in \Gamma_t$. Then we take $M := \top$

$$\begin{array}{lcl} \Gamma_s \vdash l := \star & & : M \\ \Gamma_t.(x: A) \vdash r := x & & : A \end{array}$$

DEMOT/1: By induction hypothesis, we get that A is a “source type”, $l \in \text{Tm}_\Sigma(\Gamma_t, M)$ and $r \in \text{Tm}_\Sigma(\Gamma_s.(x: M), A)$. But because of the constraint of the DEMOTE rule, it must be the case that $A = B$, in other words these type really are in the common language of Γ_s and Γ_t . Thus, we take $M' := M \rightarrow A$ and

$$\begin{array}{lcl} \Gamma_s \vdash l' := \lambda x.r & & : M \rightarrow A \\ \Gamma_t.(z: M) \vdash r' := z l & & : A \end{array}$$

DEMOT/2: In this case we keep the exact same interpolating type and interpolants.

APP/1: We are in the case of an application $t u$, with the induction hypothesis on t giving us some $A \rightarrow B$ with vocabulary in Γ_s and M_t, r_t and l_t such that $t = r_t[x := l_t]$. Because A has its vocabulary in Γ_s , by induction hypothesis we can interpolate u with a “reversed” colouring on Γ , i.e. inverting the roles of Γ_s and Γ_t . This gives us M_u, r_u and l_u such that $u = r_u[x := l_u]$. Then we take $M := M_t \times M_u$, and

$$\begin{array}{lcl} \Gamma_t \vdash l := (l_t, l_u) & & : M_t \times M_u \\ \Gamma_t.(z: M_t \times M_u) \vdash r := r_t[x := z_1] r_u[x := z_2] & & : B \end{array}$$

APP/2: This time, the induction hypothesis on t gives us some $A \rightarrow B$ with vocabulary in Γ_s and M_t, r_t and l_t such that $t = r_t[x := l_t]$. We can directly use the induction hypothesis on u , to get M_u, r_u and l_u such that $u = r_u[x := l_u]$. Then we take again $M := M_t \times M_u$, and

$$\begin{array}{lcl} \Gamma_s \vdash l := (l_t, l_u) & & : M_t \times M_u \\ \Gamma_t.(z: M_t \times M_u) \vdash r := r_t[x := z_1] r_u[x := z_2] & & : B \end{array}$$

PROJi: Here in all cases we keep the same interpolating type and left interpolant, and simply post-compose the right interpolant with the relevant projection.

It seems we do not quite construct the same interpolant as Čubrić. Indeed, in the case of a stack of applications on a source variable of type $A_1 \rightarrow \dots \rightarrow A_n \rightarrow B$, we will construct an interpolating type $(M_1 \times \dots \times M_n) \rightarrow B$, while Čubrić will rather construct $M_1 \rightarrow \dots \rightarrow M_n \rightarrow B$.

References

- [Bez+21] M. Bezem, T. Coquand, P. Dybjer, and M. Escardó. “On generalized algebraic theories and categories with families”. In: *Mathematical Structures in Computer Science* 9 (2021), pp. 1006–1023. doi: 10.1017/S0960129521000268. url: <https://www.cambridge.org/core/product/02459F7C89C72EEA9A1BC296F17B920C> (cit. on p. 1).
- [Čub94] D. Čubrić. “Interpolation property for bicartesian closed categories”. In: *Archive for Mathematical Logic* 4 (Aug. 1994), pp. 291–319. doi: 10.1007/bf01270628 (cit. on pp. 1, 3, 7).